

프로세스 관리



□ 프로세스의 개념

- 하드 디스크에 저장된 프로그램을 CPU의 명령에 따라 메모리에 로딩되어 활성화된 것을 프로세스라고 함
- 프로세스는 컴퓨터에서 실행 중인 프로그램으로, 운영체제의 관리 하에 CPU와 메모리 등의 자원을 할당받아 수행되는 작업 단위

□ 프로세스 상태 전이

- 프로세스는 운영체제의 스케줄링에 의해 상태가 전이됨
- 하나의 프로그램이 실행되면 그 프로그램에 대응되는 프로세스가 생성되어 준비 리스트의 끝에 들어가게 되고
- 준비 리스트 상의 다른 프로세스들이 CPU를 할당받아 준비 리스트를 벗어나면
- 그 프로세스는 점차 준비 리스트의 앞으로 나가게 되고 할당된 순서가 도래되면 CPU를 사용할 수 있음

□ 프로세스 번호와 작업 번호

■ 프로세스 번호

- CPU가 프로세스를 구분하기 위해 부여되는 고유 번호
- 각 프로세스가 가지는 고유의 번호를 PID라고 함

■ 작업 번호

- 현재 실행되는 백그라운드 프로세스가 CPU를 점유하여 작업을 수행 할 때의 순차 번호를 의미

□ 프로세스 서비스와 관계

■ 데몬 프로세스

- 우분투에서 특정 서비스를 제공하기 위해 존재하는 프로세스
- 평상시에는 대기 상태로 있다가 서비스 요청이 들어오면 해당 서비스를 제공

■ 부모-자식 관계의 프로세스

- 부모 프로세스는 PPID를 가지며 자식 프로세스는 PID를 가지고 있으므로 프로세스에 대한 구분이 가능
- 부모 프로세스를 종료하게 되면 종속된 자식 프로세스 또한 강제로 종료됨

■ 프로세스



부모 프로세스와 자식 프로세스의 관계

□ 고아 프로세스와 좀비 프로세스

■ 고아 프로세스

- 부모-자식 관계의 프로세스에서 자식 프로세스가 종료되지 않은 상태에서 부모 프로세스가 종료되는 경우
- 고아 프로세스의 종료는 1번 프로세스를 부모 프로세스로 의존하게 되어 작업을 무난히 마치고 프로세스를 종료하게 됨

■ 좀비 프로세스

- 부모 프로세스가 자식 프로세스의 종료를 기다린 뒤에 종료를 처리해 주는 정상적인 과정을 거치지 않고
- 부모 프로세스를 종료할 경우 발생하게 되는 프로세스를 의미

여기서 잠깐



좀비 프로세스를 제거하기 위해서는 SIGCHLD 시그널을 부모 프로세스에 보내 부모 프로세스가 자식 프로세스를 정리하도록 수행하거나 부모 프로세스 자체를 종료해야만 좀비 프로세스를 제거할 수 있습니다.

프로세스 관리 명령

□ 프로세스 상태 확인 : ps 명령

- 우분투에서 ps 명령의 옵션은 3가지 유형(유닉스, BSD, GNU)으로 지원

\$ ps

기능 현재 실행 중인 프로세스의 상태 확인

형식 ps [옵션] Enter

옵션 <유닉스 옵션>

-e : 시스템에서 실행 중인 모든 프로세스의 정보 출력

-f : 프로세스에 대한 상세한 정보 출력

-u UID : 특정 사용자에게 대한 모든 프로세스의 정보 출력

-p PID : PID로 지정한 특정 프로세스의 정보 출력

<BSD 옵션>

a : 터미널에서 실행한 프로세스의 정보 출력

u : 프로세스 소유자의 이름, CPU와 메모리 사용량 등 상세한 정보 출력

x : 시스템에서 실행 중인 모든 프로세스의 정보 출력

<GNU 옵션>

--pid PID 목록 : 목록으로 지정한 특정 PID 정보 출력

■ 프로세스 관리 명령

- 프로세스 목록 출력 : PS 명령

예제 10-01

- **Step 01** | ps 명령으로 현재 프로세스 상태정보를 출력합니다.

```
cskisa@ubuntu:~$ ps
  PID  TTY      TIME  CMD
  1958 pts/0    00:00:00  bnash
  1985 pts/0    00:00:00  ps
cskisa@ubuntu:~$
```

■ 프로세스 관리 명령

예제 10-01

- **Step 01** | ps 명령으로 현재 프로세스 상태정보를 출력합니다.

```
cskisa@ubuntu:~$ ps
  PID  TTY      TIME  CMD
  1958  pts/0    00:00:00  bnash
  1985  pts/0    00:00:00  ps
cskisa@ubuntu:~$
```

ps 명령으로 출력된 현재 프로세스의 상태정보는 다음 표와 같습니다.

표 10-1 프로세스의 항목별 상태정보

항목	의미
PID	실행 중인 프로세스를 구별하기 위한 프로세스의 고유 ID
TTY	프로세스가 시작되고 있는 현재 터미널 번호
TIME	해당 프로세스가 현재까지 사용된 CPU의 시간의 양
CMD	프로세스가 실행한 명령이 무엇인지를 알려줌

■ 프로세스 관리 명령

- **Step 02** | ps 명령과 함께 -f 옵션을 사용하여 현재 프로세스의 상세한 정보를 출력합니다.

```
cskisa@ubuntu:~$ ps -f
```

```
UID          PID    PPID  C  STIME TTY          TIME CMD
cskisa       1958    1948  0  17:47 pts/0        00:00:00 bash
cskisa       2105    1958  0  17:50 PTS/0        00:00:00 ps -f
cskisa@ubuntu:~$
```

셸 확인
echo \$SHELL

ps -f 명령으로 출력된 현재 프로세스의 상세한 상태정보의 의미는 다음 표와 같습니다.

표 10-2 프로세스의 상세한 상태정보

항목	기능	항목	기능
UID	프로세스를 실행한 사용자 ID	STIME	프로세스 시작 날짜 또는 시각
PID	실행 프로세스 번호	TTY	터미널의 종류와 번호
PPID	부모 프로세스 번호	TIME	프로세스 실행 시간
C	CPU 사용량을 % 값으로 표시	CMD	실행되고 있는 프로그램 이름

□ 프로세스 관리

- 특정 프로세스 정보를 검색할 때는 pgrep 명령을 사용
- ps 명령과 grep 명령을 하나로 통합하여 만든 명령어로 인식
- pgrep 명령은 옵션 다음에 선언한 인자와 일치하는 패턴의 프로세스를 찾아 PID를 알려줌

\$ pgrep

기능 인자와 일치하는 패턴의 프로세스 정보 출력

형식 pgrep [옵션] [인자]

옵션 -x : 인자와 정확히 일치하는 패턴의 프로세스 정보 출력

-n : 인자를 포함하고 있는 가장 최근의 프로세스 정보 출력

-u 사용자 계정 이름 : 특정 사용자 계정에 대한 모든 프로세스의 정보 출력

-l : PID와 프로세스의 이름 출력

-t term : 특정 단말기와 관련된 프로세스의 정보 출력

■ 프로세스 관리 명령

예제 10-02

- **Step 01** | pgrep 명령과 함께 -x 옵션을 선언하여 bash 프로세스의 PID 정보를 출력합니다.

```
cskisa@ubuntu:~$ pgrep -x bash
1958
cskisa@ubuntu:~$
```

- **Step 02** | pgrep 명령과 함께 -l 옵션을 선언하여 bash 프로세스의 PID와 프로세스의 이름을 출력합니다.

```
cskisa@ubuntu:~$ pgrep -l bash
1958 bash
cskisa@ubuntu:~$
```

■ 프로세스 관리 명령

- **Step 03** | `pgrep` 명령과 함께 `-l` 옵션을 선언하여 `bash` 프로세스의 PID와 프로세스의 이름을 출력합니다.

```
cskisa@ubuntu:~$ ps -fp $(pgrep -x bash)
UID          PID    PPID  C  STIME  TTY          TIME     CMD
cskisa      1958    1948  0   17:47  pts/0        00:00:00  bash
cskisa@ubuntu:~$
```

■ 프로세스 관리 명령

■ kill 명령으로 프로세스 종료

- 프로세스를 종료하는 kill 명령은 종료할 프로세스에 인자로 지정한 숫자 메시지를 시그널로 보내 해당 프로세스를 종료
- 프로세스에 보내는 시그널은 인터럽트, 프로세스 종료, 강제 종료 등의 숫자로 기능이 지정되어 있음

\$ kill

기능 프로세스 종료를 위해 지정한 시그널을 해당 프로세스에 전달

형식 kill [시그널] PID

시그널 -2 : 인터럽트 시그널 전송 (+)

-9 : 프로세스 강제 종료

-15 : 프로세스가 관련 파일을 정리 후 종료(종료되지 않는 프로세스도 존재)

■ 프로세스 관리 명령

- kill 명령으로 프로세스를 종료하기 위해 2개의 터미널 창 실행

예제 10-03

- **Step 01** | yes 명령은 단순히 yes라는 문자열을 화면에 계속 반복해서 출력하라는 의미이고 /dev/null은 아무런 반응을 하지 않는 장치를 선언한 것입니다. 강제 종료하려면 **Ctrl+Z**를 누르면 되지만 다음 과정 실습을 위해 강제 종료는 수행하지 않습니다 .

```
cskisa@ubuntu:~$ yes > /dev/null
```

① ← 무한루프 명령 수행

■ 프로세스 관리 명령

- **Step 02** | [현재 활동] - [터미널] - [새창]을 클릭하여 두 번째 터미널 창을 추가합니다.

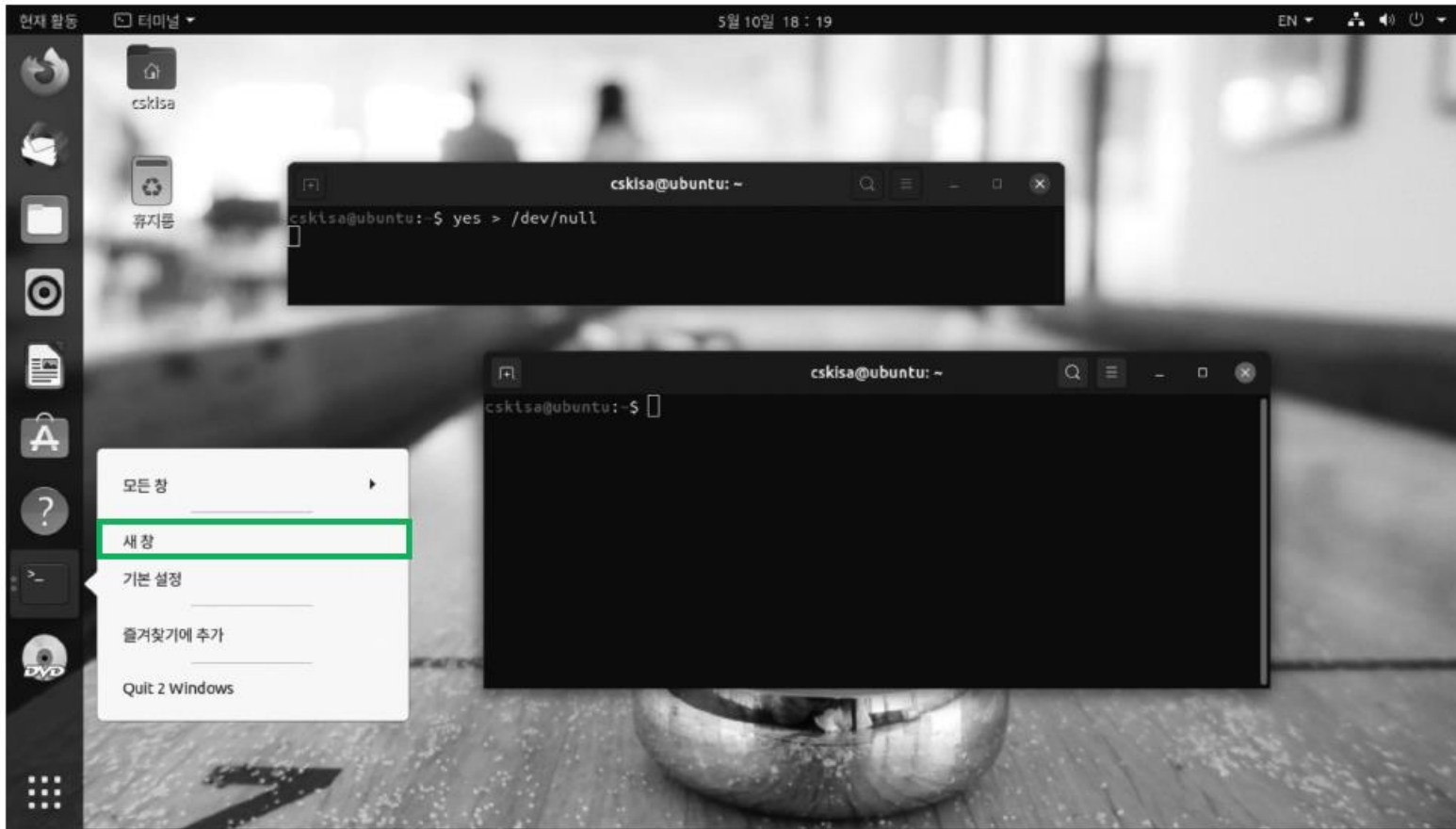


그림 10-3 터미널 창 추가

■ 프로세스 관리 명령

- **Step 03** | 첫 번째 터미널(부모 프로세스)에서 수행하고 있는 프로세스의 PID 2544를 두 번째 터미널 창에서 확인합니다. kill 명령과 시그널 -9 그리고 PID를 지정하여 자식 프로세스에서 부모 프로세스를 강제로 종료합니다.

```
cskisa@ubuntu:~$ yes > /dev/null ❶
```

← 첫 번째 터미널 창 : 부모 프로세스

```
죽었음 ❷
```

```
cskisa@ubuntu:~$
```

```
cskisa@ubuntu:~$ ps -ef | grep yes ❷
```

← 두 번째 터미널 창 : 자식 프로세스

```
cskisa      2544    2537  90 18:26 pts/0    00:00:14 yes
```

```
cskisa      2566    2556   0 18:26 pts/1    00:00:00 grep --color=auto yes
```

```
cskisa@ubuntu:~$ kill -9 2544 ❸
```

```
cskisa@ubuntu:~$
```

- pkill 명령으로 프로세스 종료
 - kill 명령과 같이 시그널을 보내는 방식은 같지만 다른 점이 있다면
 - pkill 명령은 PID를 보내는 것이 아니라 프로세스의 명령이름으로 프로세스를 찾아서 종료해 준다는 점이 kill 명령과 다른 부분임
 - pkill 명령은 같은 명령으로 수행한 프로세스를 명령 이름으로 찾아주기 때문에
 - pkill yes와 같이 한꺼번에 같은 이름의 명령을 찾아서 모두 종료할 수 있다는 편리성을 제공

도전 10-01

1. ps 명령과 옵션으로 프로세스의 상세한 정보 출력하기
2. pgrep 명령으로 bash를 실행하고 있는 프로세스의 PID 출력하기
3. ps 명령과 옵션으로 터미널에서 실행한 프로세스의 정보 출력하기

```
$ ps -f
$ pgrep -X bash
$ ps a
```

포그라운드와 백그라운드

□ 포그라운드

- 터미널 창에서 명령어를 입력하면 셸은 사용자가 입력한 명령을 수행하여 그 결과를 화면에 보여줌
- 사용자는 화면에 나타난 출력결과를 보고 다른 명령을 입력하는 대화식으로 수행하는 작업
- 즉 사용자와 상호작용을 하여 작업을 수행하도록 해주는 프로세스를 포그라운드 작업이라고 함

□ 백그라운드

- 프로세스가 실행되었지만 직접 눈으로 프로세스가 확인되지 않는 작업을 백그라운드 작업이라고 함
- 예를 들어 백신 프로그램, 서버 데몬 등과 같이 화면에 나타나 눈에 보이지는 않지만 실행되는 명령을 작업을 의미
- 이때 수행하는 프로세스를 백그라운드 프로세스라고 함

■ 포그라운드와 백그라운드

예제 10-04

- **Step 01** | 프로세스가 잠시 쉬도록 sleep 명령으로 포그라운드 작업을 수행하게 되면 현재 작업이 끝날 때까지 기다려야 하므로 다른 작업을 수행할 수 없게 됩니다.

```
cskisa@ubuntu:~$ sleep 10
```

← 10초 동안 포그라운드 작업
← 작업이 끝날 때까지 기다려야 함

- **Step 02** | 터미널 창에서 sleep 명령과 &(앰퍼샌드) 기호를 선언하여 백그라운드 작업을 30초 동안 수행합니다. 백그라운드 작업 중에는 다른 작업을 수행할 수 있습니다.

```
cskisa@ubuntu:~$ sleep 30&
[1] 2683
cskisa@ubuntu:~$
```

← 백그라운드 작업
← 프롬프트가 나타나 다른 작업 수행이 가능함

□ 작업 제어

- 포그라운드 작업을 백그라운드 작업으로 전환하거나 백그라운드 작업을 포그라운드 작업으로 전환하는 제어를 의미
- 작업 제어는 작업 전환과 작업의 일시 중지, 작업 종료로 구분되어 수행됨
- 현재 실행 중인 작업 목록을 출력할 때는 `jobs` 명령을 사용

\$ jobs

기능 작업 목록 출력

형식 jobs [%작업 번호] Enter↵

%작업 번호 %번호 : 해당 작업의 정보만 출력

 %+ 또는 %% : 작업순서가 +인 작업 정보를 출력

 %- : 작업순서가 -인 작업 정보를 출력

■ 포그라운드와 백그라운드

- 포그라운드 작업과 백그라운드 작업을 제어하는 예제 수행

예제 10-05

- **Step 01** | 터미널 창에서 백그라운드 작업과 포그라운드 작업을 다음과 같이 각각 수행합니다.

```
cskisa@ubuntu:~$ sleep 20&
[1] 2865
cskisa@ubuntu:~$ sleep 30&
[2] 2866
cskisa@ubuntu:~$ sleep 10
[1]-  완료                  sleep 20
cskisa@ubuntu:~$
```

■ 포그라운드와 백그라운드

- **Step 02** | jobs 명령으로 작업 목록을 출력해 보면 포그라운드 작업 목록은 나타나지 않고 백그라운드 작업 목록에 대해서만 출력됩니다.

```
cskisa@ubuntu:~$ jobs
```

```
[2]+  실행중                  sleep 30 &
```

```
cskisa@ubuntu:~$
```

■ 포그라운드와 백그라운드

- jobs 명령을 수행한 결과 출력된 작업 목록의 의미

표 10-3 jobs 명령의 출력항목

항목	출력	의미
작업 번호	[2]	백그라운드 작업 번호를 나타냄 ([1], [2], [3])
작업 순서	+	+ : 가장 최근에 접근한 작업
		: + 작업 전에 접근한 작업
		공백 : 그 이외의 작업
작업 상태	실행 중	실행중 : 현재 실행 (Running)
		완료됨 : 작업 정상 종료 (Done)
		종료됨 : 비정상 작업 종료 (Terminated)
		정지됨 : 작업 잠시 중단 (Stopped)
명령	sleep 30&	백그라운드로 실행 중인 명령

■ 포그라운드와 백그라운드

■ 작업 전환

- 현재 포그라운드로 실행 중인 작업을 백그라운드 작업으로 전환하거나 백그라운드 작업을 포그라운드 작업으로 전환
- 작업 전환 명령은 다음 표와 같음

표 10-4 작업 전환 명령

명령	의미
<code>Ctrl+Z</code> 또는 <code>stop [%작업 번호]</code>	포그라운드 작업을 잠시 중단
<code>bg [%작업 번호]</code>	작업 번호가 지시하는 작업을 백그라운드로 전환
<code>fg [%작업 번호]</code>	작업 번호가 지시하는 작업을 포그라운드로 전환

■ 포그라운드와 백그라운드

- 작업 전환 명령을 살펴보기 위한 예제 수행

예제 10-06

- **Step 01** | 터미널 창에서 jobs 명령으로 작업 목록을 확인합니다.

```
cskisa@ubuntu:~$ jobs
cskisa@ubuntu:~$
```

- **Step 02** | sleep 명령으로 포그라운드 작업을 수행한 다음 **Ctrl**+**Z**를 눌러 작업을 일시 중단합니다.

```
cskisa@ubuntu:~$ sleep 100          ← 포그라운드 작업 실행
^Z                                   ← Ctrl+Z를 눌러 작업을 일시 중단
[1]+  정지됨                sleep 100  ← 일시 중단된 상태
cskisa@ubuntu:~$
```

■ 포그라운드와 백그라운드

- **Step 03** | 현재 중단된 포그라운드 작업을 `bg [%작업 번호]` 명령으로 백그라운드 작업으로 전환합니다. 작업 번호는 지정하지 않고 `bg` 명령만 사용할 경우 작업순서가 +인 작업에 대해서만 적용됩니다.

```
cskisa@ubuntu:~$ bg %1          ← 백그라운드로 작업 전환
[1]+  sleep 100 &
cskisa@ubuntu:~$
```

- **Step 04** | `jobs` 명령으로 현재 수행 중인 백그라운드 작업 목록을 확인합니다.

```
cskisa@ubuntu:~$ jobs
[1]+  실행중                sleep 100 &  ← 백그라운드로 실행 중
cskisa@ubuntu:~$
```

■ 포그라운드와 백그라운드

- 포그라운드 작업 종료 : [Ctrl]+[C]
 - 포그라운드 작업은 [Ctrl]+[C]를 누르면 대부분 종료
 - 또다른 방법은 해당 프로세스의 PID를 찾아서 강제 종료
 - 단축키 [Ctrl]+[C]를 사용하여 포그라운드 작업 종료 예제 수행

예제 10-07

- **Step 01** | 터미널 창에서 sleep 명령을 수행한 다음 **[Ctrl]+[C]**를 눌러 현재 수행 중인 포그라운드 작업을 강제로 종료합니다.

```
cskisa@ubuntu:~$ sleep 80
```

```
^C
```

```
cskisa@ubuntu:~$
```

← 포그라운드로 작업 수행

← **[Ctrl]+[C]**를 눌러 작업을 강제 종료

■ 포그라운드와 백그라운드

- **Step 02** | jobs 명령으로 현재 작업 중인 목록을 확인해 보면 현재 수행 중인 포그라운드 작업이 없다는 것을 알 수 있습니다.

```
cskisa@ubuntu:~$ jobs
```

```
cskisa@ubuntu:~$
```


■ 포그라운드와 백그라운드

- 백그라운드 작업 종료 : kill
 - 백그라운드 작업은 kill 명령을 사용하여 강제 종료
 - 인자는 PID 대신 ' %작업 번호 ' 를 선언해도 됨
 - kill 명령으로 백그라운드 작업 종료 예제 수행

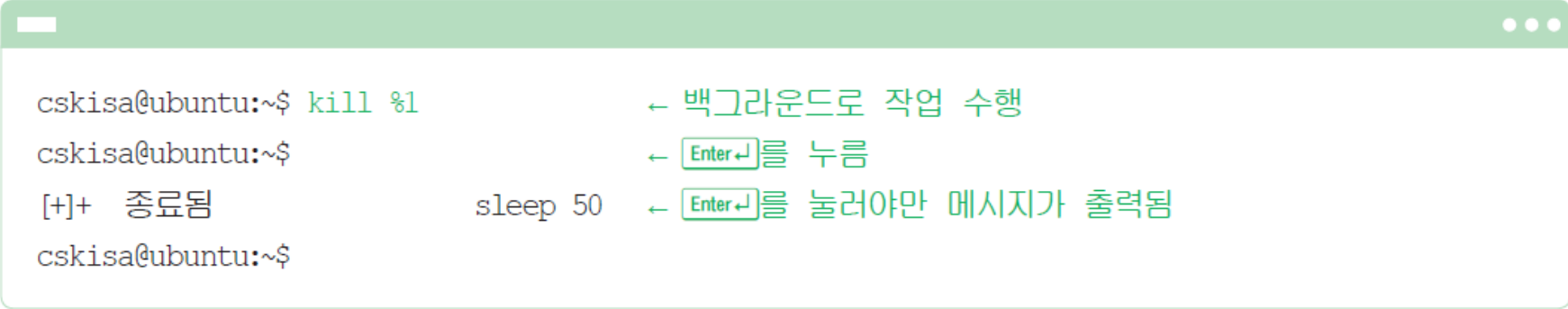
예제 10-08

- **Step 01** | 터미널 창에서 sleep 명령으로 백그라운드 작업을 수행합니다.

```
cskisa@ubuntu:~$ sleep 50&           ← 백그라운드로 작업 수행
[1] 2306
cskisa@ubuntu:~$
```

■ 포그라운드와 백그라운드

- **Step 02** | kill 명령과 작업 번호를 선언하여 현재 실행 중인 백그라운드 작업을 강제로 종료합니다.



```
cskisa@ubuntu:~$ kill %1
cskisa@ubuntu:~$
[+] 종료됨          sleep 50
cskisa@ubuntu:~$
```

← 백그라운드로 작업 수행

← Enter↵를 누름

← Enter↵를 눌러야만 메시지가 출력됨

■ 실습해보기

1. 현재 실행 중인 프로세스를 확인하시오.
2. 실시간으로 실행 중인 프로세스를 확인하시오.
3. 현재 실행 중인 프로세스 중 bash 프로세스를 검색하시오.
4. sleep 100 명령어를 백그라운드로 실행하시오.
5. 현재 백그라운드에서 실행 중인 작업을 확인하시오.
6. 실행 중인 sleep 100 프로세스를 종료하시오.
7. 현재 쉘 확인
8. 프로세스 ID 확인
9. sleep 200을 백그라운드로 실행한 뒤 종료하시오.
10. CPU를 계속 사용하는 프로세스를 생성하고 강제 종료하시오.